



LinkedIn Agent IA

Rapport Technique Complet du Projet

Type : Application Web SaaS + Extension Chrome

Version : 0.1.0

Framework : Next.js 16 + React 19 + TypeScript

Base de données : PostgreSQL (Neon DB)

IA : OpenAI GPT-4o-mini via LangChain

Date du rapport : 10 Mars 2026

Table des matières

1. Vue d'ensemble du projet
2. Architecture technique
3. Technologies utilisées
4. Structure du projet (arborescence)
5. Base de données - Schéma complet
6. API Backend - Routes détaillées
7. Composants Frontend - Détail de chaque fichier
8. Bibliothèque lib/ - Logique métier
9. Extension Chrome
10. Agent IA - Fonctionnement détaillé
11. Système de Rate Limiting
12. Déploiement (Docker)
13. Composants UI réutilisables
14. Fonctionnalités clés
15. Sécurité et bonnes pratiques

1. Vue d'ensemble du projet

LinkedIn Agent IA est une plateforme SaaS complète de prospection LinkedIn automatisée, pilotée par une intelligence artificielle (GPT-4o-mini). Elle combine une application web Next.js, une extension Chrome et un agent IA pour automatiser la recherche de prospects, l'envoi de messages et le suivi des campagnes.

Objectif

Permettre aux professionnels B2B d'automatiser leur prospection LinkedIn de manière intelligente, en respectant les limites LinkedIn, avec un système d'approbation humaine pour chaque action sensible.

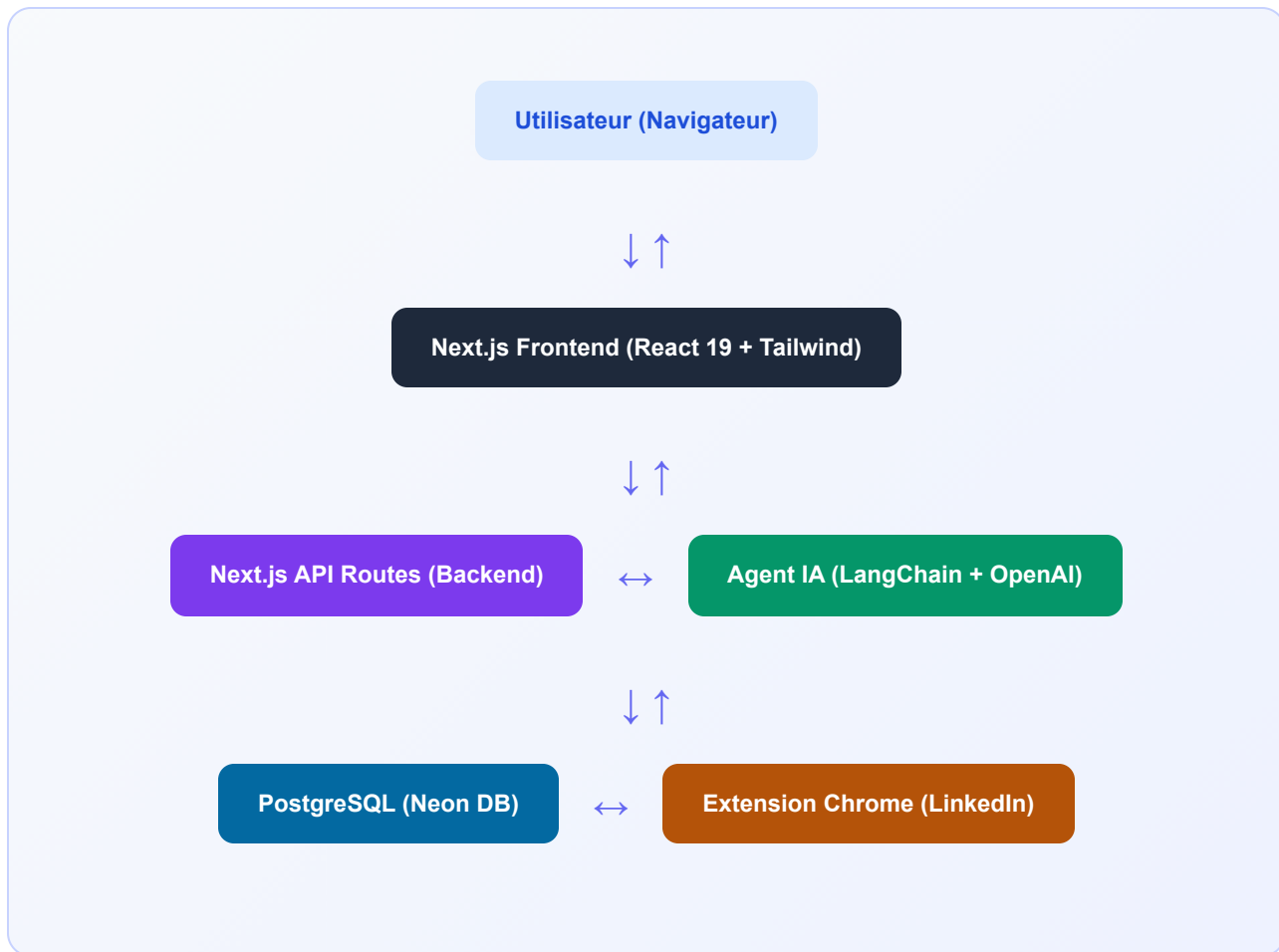
Chiffres clés du projet



Les 3 piliers du système

PILIER	DESCRIPTION	TECHNOLOGIES
Application Web	Dashboard de gestion : campagnes, messages, prospects, statistiques, paramètres	Next.js 16, React 19, Tailwind CSS 4, Framer Motion
Agent IA	Chatbot intelligent qui exécute des actions LinkedIn via des outils (tools)	LangChain, OpenAI GPT-4o-mini, Tool Calling
Extension Chrome	Exécute les actions approuvées directement sur LinkedIn (recherche, connexion, message)	Manifest V3, Content Scripts, Background Service Worker

2. Architecture technique



Flux de données principal

1. L'utilisateur interagit avec le **Dashboard** ou le **Chat IA**
2. Les requêtes passent par les **API Routes Next.js**
3. L'agent IA utilise des **Tools LangChain** pour créer des actions dans la **queue**
4. L'utilisateur **approuve ou rejette** chaque action dans la file d'approbation
5. L'**extension Chrome** récupère les actions approuvées et les exécute sur LinkedIn
6. Les résultats sont renvoyés à la base de données et affichés dans le dashboard

3. Technologies utilisées

Frontend

Next.js 16.1.6

Framework React fullstack avec App Router

React 19.2.3

Bibliothèque UI avec hooks et composants fonctionnels

TypeScript 5

Typage statique pour la sûreté du code

Tailwind CSS 4

Framework CSS utility-first

Framer Motion 12

Animations et transitions fluides

Recharts 3.7

Graphiques et visualisations de données

Lucide React

Icônes SVG légères et modulaires

Radix UI

Composants accessibles (Dialog, Tabs, Select, Dropdown)

CVA + clsx

Gestion de variantes de classes CSS

Backend & Base de données

Next.js API Routes

Endpoints REST intégrés au projet

PostgreSQL (Neon DB)

Base relationnelle serverless dans le cloud

node-postgres (pg)

Client PostgreSQL natif pour Node.js

Intelligence Artificielle

LangChain 1.2.30

Framework d'orchestration IA avec tools

@langchain/openai

Intégration OpenAI pour GPT-4o-mini

OpenAI GPT-4o-mini

Modèle de langage pour le chat et les décisions

Déploiement & DevOps

Docker

Docker Compose

Orchestration des services

ESLint 9

Linting et qualité du code

Conteneurisation multi-stage (Alpine)		
--	--	--

4. Structure du projet

```

linkedin-agent-frontend/ |— app/ # Next.js App Router | |— layout.tsx # Layout
racine (metadata, fonts) | |— page.tsx # Page principale (Dashboard) | |—
globals.css # Styles globaux + Tailwind | |— api/ # 11 groupes de routes API |
|— agent/chat/ # Chat avec l'agent IA | |— campaigns/ # CRUD campagnes | |—
linkedin-actions/ # Queue d'actions + approbation | |— linkedin-auth/ #
Authentification LinkedIn | |— messages/ # Gestion des messages | |—
notifications/ # Notifications temps réel | |— prospects/ # CRUD prospects | |—
stats/ # Statistiques dashboard | |— templates/ # Templates de messages | |—
test-db/ # Test de connexion DB | |— users/ # Gestion utilisateurs | |—
components/ # 17 composants React | |— Dashboard.tsx # Page d'accueil avec KPIs |
|— Sidebar.tsx # Navigation latérale | |— Header.tsx # Barre supérieure | |—
StatsCards.tsx # Cartes de statistiques | |— PerformanceCharts.tsx # Graphiques
(Recharts) | |— RecentActivity.tsx # Activité récente | |— ProspectsPipeline.tsx
# Pipeline prospects (5 étapes) | |— Campaigns.tsx # Gestion des campagnes | |—
Messages.tsx # Boîte de réception messages | |— ApprovalQueue.tsx # File
d'approbation des actions | |— AgentChat.tsx # Chat avec l'agent IA | |—
Settings.tsx # Paramètres complets | |— LinkedInAccount.tsx # Connexion compte
LinkedIn | |— NotificationPanel.tsx # Panneau de notifications | |— ui/ # 9
composants UI réutilisables | |— button.tsx | |— card.tsx | |— badge.tsx | |—
input.tsx | |— tabs.tsx | |— switch.tsx | |— slider.tsx | |— separator.tsx |
|— progress-bar.tsx | |— lib/ # Logique métier | |— db.ts # Connexion
PostgreSQL + CRUD | |— agent.ts # Classe LinkedInAgent (LangChain) | |— agent-
config.ts # Config IA + limites LinkedIn | |— tools.ts # 10+ outils LangChain |
|— rate-limiter.ts # Limiteur de débit LinkedIn | |— utils.ts # Utilitaires (cn,
clsx) | |— db/ # Base de données | |— schema.sql # Schéma complet (9 tables) |
|— seed.sql # Données de test | |— linkedin-chrome-extension/ # Extension Chrome
| |— manifest.json # Manifest V3 | |— background.js # Service Worker | |—
content.js # Scripts injectés sur LinkedIn | |— popup.html / popup.js # Interface
popup | |— styles.css # Styles de l'extension | |— package.json # Dépendances
NPM |— Dockerfile # Image Docker multi-stage |— docker-compose.yml #
Orchestration |— tsconfig.json # Configuration TypeScript

```

5. Base de données - Schéma complet

La base de données PostgreSQL est hébergée sur **Neon DB** (serverless). Elle contient 9 tables avec des index pour les performances. Fichier : `db/schema.sql`

Table `campaigns`

Stocke les campagnes de prospection avec leurs critères de ciblage et métriques.

COLONNE	TYPE	DESCRIPTION
id	SERIAL PK	Identifiant unique
name	VARCHAR(255)	Nom de la campagne
status	VARCHAR(50)	draft, active, paused, completed
target, template	VARCHAR(255)	Cible et template utilisé
description	TEXT	Description détaillée
industry, location, company_size, target_role	VARCHAR	Critères de ciblage
template_invitation, template_followup	TEXT	Templates de messages
contacted, replied, clicked, converted	INTEGER	Compteurs de performance
created_at, updated_at	TIMESTAMP	Dates de création/modification

Table `prospects`

Informations sur les prospects LinkedIn identifiés et qualifiés.

COLONNE	TYPE	DESCRIPTION
id	SERIAL PK	Identifiant unique
linkedin_url	VARCHAR(500) UNIQUE	URL du profil LinkedIn

name, role, company	VARCHAR	Infos du prospect
industry, location, company_size	VARCHAR	Critères de qualification
email, phone	VARCHAR	Coordonnées
score	INTEGER	Score de qualification (0-100)
status	VARCHAR(50)	new, contacted, replied, qualified, converted
notes	TEXT	Notes personnelles
last_seen	TIMESTAMP	Dernière activité

Table messages

Tous les messages envoyés aux prospects dans le cadre des campagnes.

COLONNE	TYPE	DESCRIPTION
id	SERIAL PK	Identifiant
campaign_id	INTEGER FK	Référence à la campagne
prospect_id	INTEGER FK	Référence au prospect
recipient_name, recipient_role, recipient_company	VARCHAR	Infos du destinataire
message_text	TEXT	Contenu du message
message_type	VARCHAR(50)	connection, followup, message
status	VARCHAR(50)	sent, delivered, read, replied

Table linkedin_actions_queue

File d'attente des actions LinkedIn. C'est le pont entre l'agent IA et l'extension Chrome.

COLONNE	TYPE	DESCRIPTION
id	SERIAL PK	Identifiant de l'action
action_type	VARCHAR(50)	search, visit_profile, send_connection, send_message

target_url	VARCHAR(500)	URL LinkedIn cible
target_name	VARCHAR(255)	Nom du prospect cible
payload	JSONB	Données de l'action (mots-clés, message, note...)
status	VARCHAR(50)	pending_approval, approved, completed, failed, rejected
result	JSONB	Résultat de l'exécution
campaign_id, prospect_id	INTEGER FK	Références optionnelles

Autres tables

TABLE	DESCRIPTION
users	Utilisateurs de la plateforme (nom, email, company, connexion LinkedIn, settings JSONB)
agent_chat_history	Historique des conversations avec l'agent IA (role, content, context JSONB)
templates	Templates de messages réutilisables (nom, tag, texte, taux de conversion)
notifications	Notifications utilisateur (type, titre, message, read, data JSONB)
scheduled_followups	Follow-ups planifiés (prospect, campagne, date prévue, statut)

Index de performance

12 index sont créés pour optimiser les requêtes fréquentes : `idx_notifications_user_id` , `idx_campaigns_status` , `idx_messages_campaign_id` , `idx_prospects_status` , `idx_prospects_score` , `idx_prospects_industry` , etc.

6. API Backend - Routes détaillées

Toutes les routes API sont définies dans `app/api/` en utilisant le système de fichiers de Next.js App Router. Chaque `route.ts` exporte des fonctions GET, POST, PATCH, DELETE.

ROUTE	MÉTHODES	DESCRIPTION	FICHIER
<code>/api/agent/chat</code>	POST	Envoie un message à l'agent IA, reçoit la réponse. Utilise LangChain avec tool calling.	<code>agent/chat/route.ts</code>
<code>/api/campaigns</code>	GET POST	Liste et crée des campagnes. Supporte la pagination.	<code>campaigns/route.ts</code>
<code>/api/campaigns/[id]</code>	GET PATCH DELETE	Détail, mise à jour et suppression d'une campagne.	<code>campaigns/[id]/route.ts</code>
<code>/api/linkedin-actions/approval</code>	GET POST DELETE	File d'approbation : liste les actions par statut (pending, approved, failed_rejected), approuve/rejette, supprime. Retourne les métriques complètes.	<code>linkedin-actions/approval/route.ts</code>
<code>/api/linkedin-actions/poll</code>	GET POST	Endpoint pour l'extension Chrome :	<code>linkedin-actions/poll/route.ts</code>

récupère les
actions
approuvées,
rapporte les
résultats.

<code>/api/messages</code>	GET POST	Liste et crée des messages. Filtrable par campagne.	<code>messages/route.ts</code>
<code>/api/messages/templates</code>	GET POST	CRUD pour les templates de messages.	<code>messages/templates/route.ts</code>
<code>/api/prospects</code>	GET POST	Liste les prospects avec filtres (status, industry, score). Crée un nouveau prospect.	<code>prospects/route.ts</code>
<code>/api/stats</code>	GET	Statistiques globales du dashboard (totaux, graphiques, tendances).	<code>stats/route.ts</code>
<code>/api/notifications</code>	GET POST PATCH	Récupère, crée et marque les notifications comme lues.	<code>notifications/route.ts</code>
<code>/api/users</code>	GET PATCH	Profil et paramètres utilisateur.	<code>users/route.ts</code>

7. Composants Frontend - Détail de chaque fichier

Dashboard.tsx 6.4 KB

Rôle : Page d'accueil principale. Orchestre l'affichage de tous les sous-composants du tableau de bord.

Fonctionnalités :

- Gestion de la navigation entre sections (Dashboard, Campagnes, Messages, Prospects, Approbations, Paramètres)
- Affichage conditionnel des composants selon la section active
- Intégration du Sidebar, Header, StatsCards, PerformanceCharts, RecentActivity, ProspectsPipeline

Hooks : `useState` pour la navigation

Sidebar.tsx 4.9 KB

Rôle : Barre de navigation latérale avec les liens vers chaque section.

Fonctionnalités :

- Icônes Lucide pour chaque section
- État actif avec indicateur visuel
- Logo et branding en haut

Header.tsx 1.1 KB

Rôle : Barre supérieure avec recherche et notifications.

StatsCards.tsx 6 KB

Rôle : Affichage des KPIs principaux sous forme de cartes colorées avec animations.

Fonctionnalités :

- 4+ cartes de statistiques (prospects, messages, taux de réponse, conversions)
- Animation de comptage (count-up) pour les chiffres
- Indicateurs de tendance (hausse/baisse) avec pourcentage

- Couleurs sémantiques configurables via `colorConfig`

PerformanceCharts.tsx 8.7 KB

Rôle : Graphiques de performance utilisant Recharts.

Fonctionnalités :

- Graphiques en ligne/barres pour les tendances
- Filtrage par période (7j, 30j, 90j)
- Tooltips interactifs avec détails

Librairie : `recharts` (AreaChart, BarChart, ResponsiveContainer)

RecentActivity.tsx 8.9 KB

Rôle : Flux d'activité récente avec timeline.

Fonctionnalités : Liste des dernières actions (connexions acceptées, messages lus, réponses reçues) avec horodatage.

ProspectsPipeline.tsx 18.7 KB

Rôle : Pipeline visuel des prospects en 5 étapes (type Kanban).

Étapes :

- **Identifiés** : Prospects trouvés via recherche
- **Contactés** : Invitation envoyée
- **Réponses** : Ont répondu au message
- **Qualifiés** : Prospect validé comme opportunité
- **Convertis** : Rendez-vous ou vente conclue

Fonctionnalités : Cartes détaillées par prospect, score, couleurs sémantiques par étape.

7. Composants Frontend (suite)

Campaigns.tsx 52 KB - Le plus gros composant

Rôle : Gestion complète des campagnes de prospection.

Fonctionnalités :

- Liste des campagnes avec filtrage par statut (brouillon, active, en pause, terminée)
- Formulaire de création de campagne (nom, cible, industrie, taille entreprise, localisation)
- Templates d'invitation et de follow-up intégrés
- Détail d'une campagne avec métriques (contactés, réponses, clics, conversions)
- Actions : démarrer, mettre en pause, supprimer
- CRUD complet connecté à l'API `/api/campaigns`

Hooks : `useState` , `useEffect` , `useCallback`

Connecté à la DB : Oui, via `/api/campaigns`

Messages.tsx 27.9 KB

Rôle : Boîte de réception des messages LinkedIn envoyés aux prospects.

Fonctionnalités :

- Liste des conversations (type WhatsApp/Messenger)
- Vue détaillée d'une conversation avec historique
- Envoi de nouveaux messages
- Templates de messages réutilisables
- Statuts : envoyé, livré, lu, répondu, en attente
- Recherche dans les conversations

ApprovalQueue.tsx 20.2 KB

Rôle : File d'approbation pour les actions générées par l'agent IA.

Fonctionnalités :

- 5 cartes de statistiques : En attente, Approuvées, Terminées, Échouées/Rejetées, Total
- Filtrage par statut en cliquant sur les cartes
- Bouton Rafraîchir manuel avec état de chargement
- Cartes détaillées par action (type, cible, URL, payload, date)

- Boutons Approuver / Rejeter pour les actions en attente
- Bouton Réessayer pour les actions échouées ou rejetées
- Bouton "Tout réessayer" (bulk retry) pour les échouées/rejetées
- Carte d'information "Comment ça marche"

Connecté à la DB : Oui, via `/api/linkedin-actions/approval`

AgentChat.tsx 10 KB

Rôle : Interface de chat avec l'agent IA en bas à droite de l'écran.

Fonctionnalités :

- Chat flottant (bulle en bas à droite, ouvrable/fermable)
- Envoi de messages à l'agent IA via `/api/agent/chat`
- Affichage des réponses avec formatage Markdown
- État de chargement animé pendant la réponse
- Suggestions de messages rapides
- Défilement automatique vers le bas

Connecté à la DB : Oui, via `/api/agent/chat` → LangChain → Tools → DB

Settings.tsx 111.8 KB - Fichier le plus volumineux

Rôle : Page complète de paramètres de l'application.

Sections :

- Profil utilisateur (nom, email, entreprise)
- Connexion LinkedIn (OAuth)
- Configuration de l'agent IA (modèle, température, prompts)
- Limites LinkedIn (connexions/jour, messages/jour, délais)
- Notifications (email, push, fréquence)
- Apparence (thème, langue)
- Sécurité (changement mot de passe, 2FA)
- Données et confidentialité (export, suppression)

LinkedInAccount.tsx 20.5 KB

Rôle : Gestion de la connexion au compte LinkedIn.

Fonctionnalités : Statut de connexion, déconnexion, informations du compte lié.

NotificationPanel.tsx 8 KB

Rôle : Panneau latéral de notifications.

Fonctionnalités : Liste des notifications, marquer comme lu, types (message, réponse, conversion, connexion).

8. Bibliothèque lib/ - Logique métier

db.ts 6.2 KB

Rôle : Couche d'accès à la base de données PostgreSQL via `node-postgres`.

Contenu :

- `pool` – Pool de connexions PostgreSQL (Neon DB avec SSL)
- `query(text, params)` – Fonction générique d'exécution SQL
- `getClient()` – Client pour les transactions
- `testConnection()` – Test de connectivité
- Fonctions CRUD : `getCampaigns`, `createCampaign`, `updateCampaign`, `deleteCampaign`
- Fonctions messages : `getMessages`, `createMessage`
- Fonctions notifications : `getNotifications`, `createNotification`, `markAsRead`, `markAllAsRead`
- Fonctions prospects : `updateProspectLastSeen`, `getOnlineProspectsCount`

agent.ts 5.2 KB

Rôle : Classe `LinkedInAgent` – orchestrateur de l'agent IA avec LangChain.

Fonctionnement :

1. Reçoit un message utilisateur + contexte optionnel
2. Construit les messages LangChain (SystemMessage + historique + HumanMessage)
3. Appelle GPT-4o-mini via `ChatOpenAI` avec tool binding
4. Si le modèle décide d'utiliser un outil : exécute le tool et fait un deuxième appel
5. Gère l'historique de conversation (max 20 messages)

Pattern : Singleton via `getLinkedInAgent()`

agent-config.ts 3.4 KB

Rôle : Configuration de l'agent IA et des limites LinkedIn.

Contenu :

- `OPENAI_CONFIG` – Modèle (gpt-4o-mini), température (0.7), maxTokens (4096)
- `SYSTEM_PROMPTS` – 3 prompts spécialisés : prospecting, messaging, campaign
- `MEMORY_CONFIG` – Configuration de la mémoire (max 20, TTL 30min)

- `LINKEDIN_LIMITS` – Limites sécurisées (20 connexions/j, 50 messages/j, 80 visites/j, 30 recherches/j)

tools.ts 32.2 KB

Rôle : Définition de tous les outils LangChain que l'agent IA peut utiliser.

Catégories d'outils :

OUTIL	DESCRIPTION
<code>linkedin_search</code>	Rechercher des profils LinkedIn (mots-clés, rôle, localisation, industrie)
<code>linkedin_visit_profile</code>	Visiter un profil pour récupérer les infos
<code>linkedin_send_connection</code>	Envoyer une demande de connexion avec note personnalisée
<code>linkedin_send_message</code>	Envoyer un message à un contact
<code>create_campaign</code>	Créer une nouvelle campagne de prospection
<code>get_prospects</code>	Récupérer la liste des prospects
<code>rate_limit_status</code>	Vérifier les limites LinkedIn restantes
<code>schedule_followup</code>	Planifier un follow-up

Chaque outil utilise `DynamicStructuredTool` de LangChain avec un schéma Zod pour la validation des paramètres. Toutes les actions LinkedIn sont mises en **queue d'approbation** (pas d'exécution directe).

rate-limiter.ts 7.6 KB

Rôle : Système de rate limiting pour respecter les limites LinkedIn.

Vérifications :

- Limite quotidienne par type d'action (ex: 20 connexions/jour)
- Limite horaire par type d'action (ex: 5 connexions/heure)
- Délai minimum entre chaque action (ex: 90s entre connexions)
- Comptage via requêtes SQL sur la table `linkedin_actions_queue`

utils.ts 166 B

Rôle : Fonction utilitaire `cn()` pour fusionner des classes Tailwind CSS (`clsx` + `tailwind-merge`).

9. Extension Chrome

L'extension Chrome est le bras exécuteur du système. Elle récupère les actions approuvées depuis l'API et les exécute directement sur les pages LinkedIn.

Fichiers de l'extension

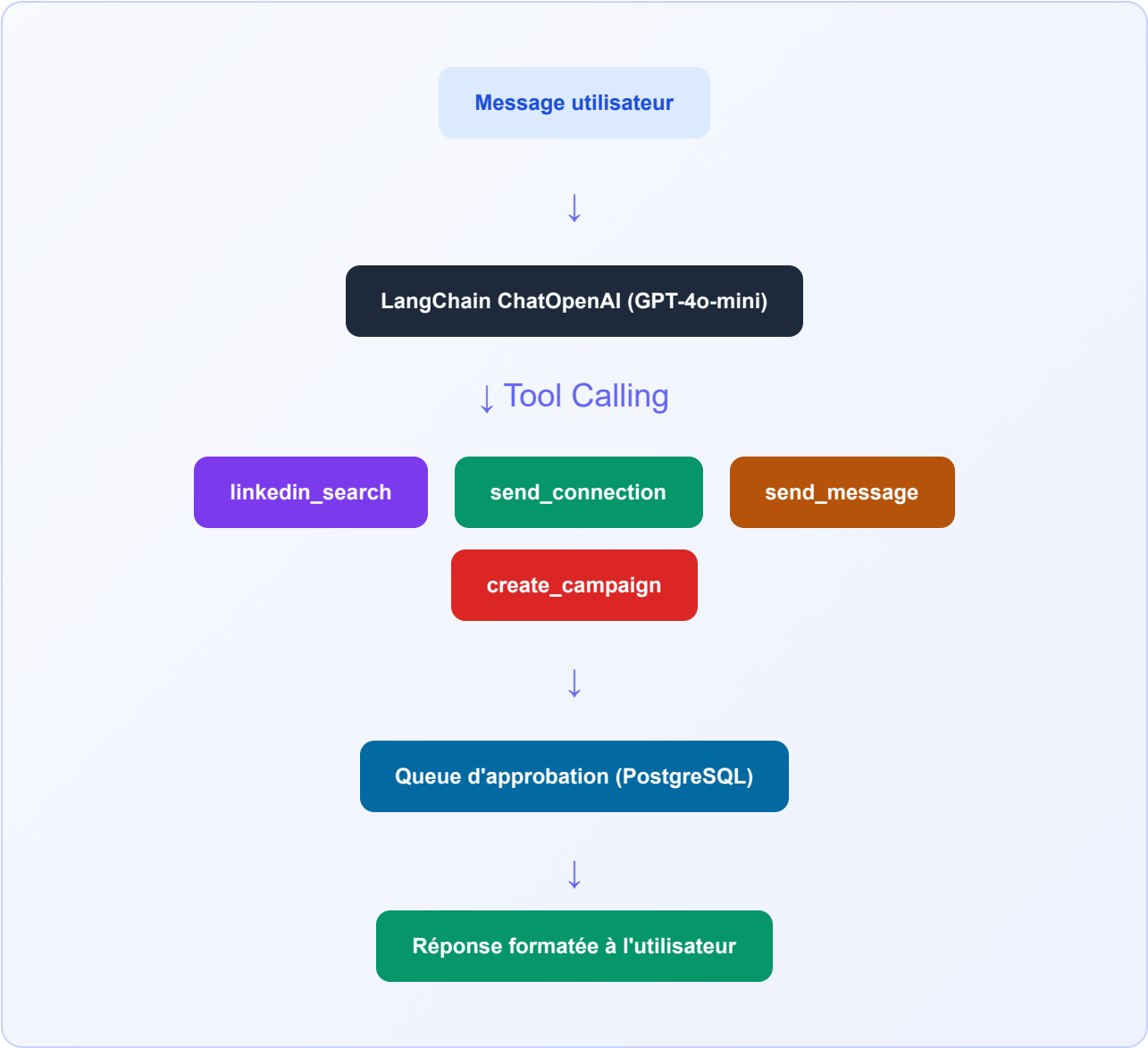
FICHIER	TAILLE	DESCRIPTION
manifest.json	926 B	Configuration Manifest V3. Permissions : <code>activeTab</code> , <code>storage</code> , <code>alarms</code> . Host permissions : <code>linkedin.com</code> , <code>localhost:3000</code> .
background.js	8.8 KB	Service Worker. Gère le polling de l'API (<code>/api/linkedin-actions/poll</code>), récupère les actions approuvées, envoie les instructions aux content scripts.
content.js	12.5 KB	Script injecté sur les pages LinkedIn. Exécute les actions : remplir la barre de recherche, cliquer sur "Se connecter", envoyer des messages, extraire les données de profils.
popup.html	6.6 KB	Interface popup de l'extension (statut de connexion, actions en cours, bouton on/off).
popup.js	4 KB	Logique de la popup (affichage du statut, toggle activation).
styles.css	1.1 KB	Styles de l'extension injectés dans LinkedIn.

Flux d'exécution

- `background.js` poll l'API toutes les 10 secondes pour récupérer les actions approuvées
- Quand une action est trouvée, il envoie un message au `content.js`
- `content.js` manipule le DOM de LinkedIn pour exécuter l'action
- Le résultat (succès/erreur) est renvoyé à l'API via `POST /api/linkedin-actions/poll`
- Le statut de l'action passe de "approved" à "completed" ou "failed"

10. Agent IA - Fonctionnement détaillé

Architecture de l'agent



Prompts système

L'agent dispose de 3 profils spécialisés :

PROFIL	CAPACITÉS
Prospecting	Analyser des profils, évaluer le potentiel, scoring, stratégies par industrie

Messaging	Rédiger des messages, gérer les objections, relances (J+3, J+7, J+14), CTA
Campaign	Concevoir des campagnes, définir le ciblage, créer des séquences drip, optimiser les KPIs

Règles de sécurité LinkedIn

ACTION	LIMITE/JOUR	LIMITE/HEURE	DÉLAI ENTRE ACTIONS
Connexions	20	5	1 min 30s
Messages	50	10	1 min
Visites de profil	80	20	15s
Recherches	30	~4	30s
Follow-ups	30	~4	5s

11. Système de Rate Limiting

Le rate limiter (`lib/rate-limiter.ts`) protège le compte LinkedIn de l'utilisateur en vérifiant les limites AVANT chaque action.

Fonctionnement

1. Avant chaque action, `checkRateLimit(actionType)` est appelé
2. Compte le nombre d'actions du même type effectuées aujourd'hui et dans la dernière heure
3. Compare avec les limites définies dans `LINKEDIN_LIMITS`
4. Si la limite est atteinte, l'action est refusée avec un message explicatif
5. Vérifie aussi le délai minimum depuis la dernière action du même type

Retour de la fonction

La fonction retourne un objet contenant :

- `allowed` : booléen indiquant si l'action est autorisée
- `reason` : raison du refus si applicable
- `dailyUsed / dailyMax` : usage quotidien
- `hourlyUsed / hourlyMax` : usage horaire
- `remainingToday` : actions restantes aujourd'hui

12. Déploiement (Docker)

Dockerfile (multi-stage)

ÉTAPE	IMAGE	ACTION
deps	node:20-alpine	Installation des dépendances (<code>npm ci</code>)
builder	node:20-alpine	Build de l'application (<code>npm run build</code>)

runner

node:20-alpine

Image de production légère (standalone output, port 3000)

docker-compose.yml

Orchestre le service `linkedin-agent-frontend` sur le port 3000. La base de données est externe (Neon DB cloud).

Variables d'environnement

VARIABLE	DESCRIPTION
<code>DATABASE_URL</code>	URL de connexion PostgreSQL (Neon DB)
<code>OPENAI_API_KEY</code>	Clé API OpenAI pour l'agent IA
<code>OPENAI_MODEL</code>	Modèle (défaut: gpt-4o-mini)

13. Composants UI réutilisables

Le dossier `components/ui/` contient 9 composants primitifs basés sur Radix UI et CVA (class-variance-authority), suivant le pattern shadcn/ui.

COMPOSANT	FICHIER	DESCRIPTION	VARIANTES
Button	<code>button.tsx</code>	Bouton avec multiples variantes	default, destructive, outline, secondary, ghost, link + tailles sm, default, lg, icon
Card	<code>card.tsx</code>	Conteneur de carte	Card, CardHeader, CardTitle, CardDescription, CardContent, CardFooter
Badge	<code>badge.tsx</code>	Étiquette colorée	default, secondary, destructive, outline
Input	<code>input.tsx</code>	Champ de saisie	Styles Tailwind avec focus states
Tabs	<code>tabs.tsx</code>	Système d'onglets	Tabs, TabsList, TabsTrigger, TabsContent (Radix UI)
Switch	<code>switch.tsx</code>	Interrupteur on/off	Radix UI Switch avec styles Tailwind
Slider	<code>slider.tsx</code>	Curseur de valeur	Radix-style avec track et thumb
Separator	<code>separator.tsx</code>	Séparateur horizontal/vertical	Orientations horizontal, vertical
ProgressBar	<code>progress-bar.tsx</code>	Barre de progression	Avec pourcentage et couleurs dynamiques

14. Fonctionnalités clés

1. Prospection automatisée

- Recherche de prospects par mots-clés, industrie, localisation, taille d'entreprise
- Pipeline visuel en 5 étapes (Identifié, Contacté, Réponse, Qualifié, Converti)
- Scoring automatique des prospects

2. Agent IA conversationnel

- Chat en langage naturel avec l'agent
- Exécution d'actions via des outils LangChain (tool calling)
- Génération de messages personnalisés
- Suggestions proactives

3. Système d'approbation (Human-in-the-Loop)

- Aucune action LinkedIn n'est exécutée sans approbation humaine
- File d'attente avec détails complets de chaque action
- Possibilité de réessayer les actions échouées ou rejetées
- Retry en masse (bulk retry)

4. Campagnes de prospection

- Création de campagnes ciblées avec templates
- Suivi des KPIs (contactés, réponses, conversions)
- Gestion du cycle de vie (brouillon → active → en pause → terminée)

5. Messagerie LinkedIn

- Boîte de réception des conversations
- Templates de messages réutilisables
- Suivi des statuts (envoyé, livré, lu, répondu)

6. Analytics et rapports

- Dashboard avec KPIs en temps réel
- Graphiques de performance (Recharts)

- Activité récente

7. Extension Chrome

- Exécution automatisée des actions approuvées sur LinkedIn
- Extraction de données de profils
- Communication bidirectionnelle avec l'API

15. Sécurité et bonnes pratiques

ASPECT	MISE EN ŒUVRE
Approbation humaine	Toutes les actions LinkedIn passent par une file d'approbation. Aucune action automatique non validée.
Rate Limiting	Limites quotidiennes et horaires pour chaque type d'action, avec délais entre actions pour éviter les bans LinkedIn.
Variables d'environnement	Clés API et URL de base de données stockées dans <code>.env.local</code> , jamais dans le code source.
SSL/TLS	Connexion à Neon DB via SSL (<code>sslmode=require</code>).
Requêtes paramétrées	Toutes les requêtes SQL utilisent des paramètres (<code>\$1, \$2...</code>) pour éviter les injections SQL.
TypeScript strict	Typage fort avec interfaces pour toutes les structures de données.
Validation Zod	Les paramètres des outils LangChain sont validés par des schémas Zod.
Docker multi-stage	Image de production minimale (Alpine, utilisateur non-root).
Connection pooling	Pool de connexions PostgreSQL pour éviter les fuites de connexions.
Gestion d'erreurs	Try/catch systématique avec messages d'erreur clairs dans toutes les routes API.

LinkedIn Agent IA — Rapport Technique

Généré le 10 Mars 2026

Version 0.1.0 | Next.js 16 + React 19 + LangChain + PostgreSQL